

UGV navigation using A* algorithm

Objective: A UGV must travel from a starting cell to a goal cell in a 70×70 km grid while avoiding obstacles.

```
import random
import heapq
import time
import matplotlib.pyplot as plt
```

```
SIZE = 70
```

```
START = (0, 0)
GOAL = (69, 69)
```

```
def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def generate_grid(density):
    grid = [[0 for _ in range(SIZE)] for _ in range(SIZE)]

    for i in range(SIZE):
        for j in range(SIZE):
            if (i, j) != START and (i, j) != GOAL:
                if random.random() < density:
                    grid[i][j] = 1

    return grid
```

```
def astar(grid):
    open_list = [(0, START)]

    g_cost = {START: 0}
    parent = {}

    visited = set()
    nodes_explored = 0

    while open_list:
        f, current = heapq.heappop(open_list)

        if current in visited:
            continue

        visited.add(current)
        nodes_explored += 1

        if current == GOAL:
            path = []

            while current in parent:
                path.append(current)
                current = parent[current]

            path.append(START)
            path.reverse()

            return path, nodes_explored

        x, y = current

        neighbors = [
            (x + 1, y),
            (x - 1, y),
            (x, y + 1),
            (x, y - 1)
        ]

        for nx, ny in neighbors:
```

```

        if 0 <= nx < SIZE and 0 <= ny < SIZE:

            if grid[nx][ny] == 1:
                continue

            neighbor = (nx, ny)

            new_cost = g_cost[current] + 1

            if new_cost < g_cost.get(neighbor, float('inf')):

                g_cost[neighbor] = new_cost
                parent[neighbor] = current

                f_cost = new_cost + heuristic(neighbor, GOAL)

                heapq.heappush(
                    open_list,
                    (f_cost, neighbor)
                )

    return None, nodes_explored

```

```

def display_grid(grid, path, density):

    plt.figure(figsize=(8, 8))

    plt.imshow(grid, cmap="Greys", origin="upper")

    if path:

        x = [p[1] for p in path]
        y = [p[0] for p in path]

        plt.plot(x, y, color="blue", linewidth=2, label="UGV Path")

    plt.scatter(START[1], START[0], color="green",
                s=100, label="Start")

    plt.scatter(GOAL[1], GOAL[0], color="red",
                s=100, label="Goal")

    plt.title(f"A* UGV Navigation - Density {density * 100:.0f}%")

    plt.legend()
    plt.show()

```

```

random.seed(42)

density_levels = {
    "Low": 0.10,
    "Medium": 0.25,
    "High": 0.40
}

for level, density in density_levels.items():

    grid = generate_grid(density)

    start_time = time.time()

    path, explored = astar(grid)

    end_time = time.time()

    print("\nObstacle Density:", level)

    if path:

        print("Path Found")
        print("Path Length:", len(path) - 1, "km")
        print("Nodes Explored:", explored)
        print("Execution Time:", round(end_time - start_time, 5), "seconds")

        display_grid(grid, path, density)

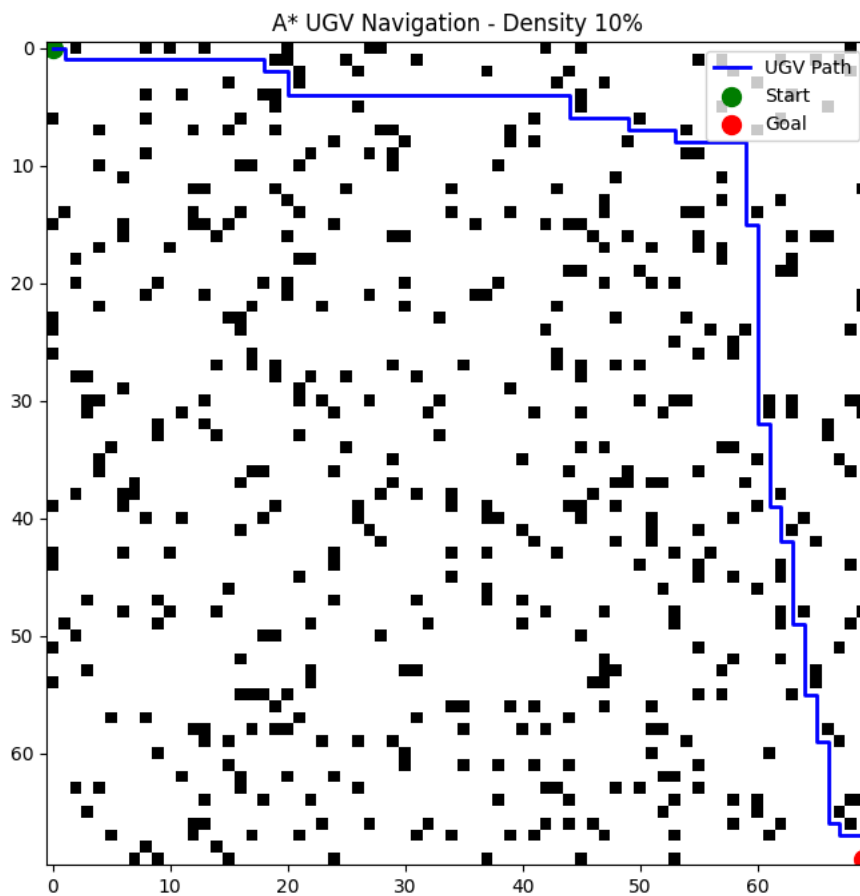
    else:

        print("No path found")

```

```
print("Nodes Explored:", explored)
print("Execution Time:", round(end_time - start_time, 5), "seconds")
```

Obstacle Density: Low
Path Found
Path Length: 138 km
Nodes Explored: 3727
Execution Time: 0.02246 seconds



Obstacle Density: Medium
No path found
Nodes Explored: 1
Execution Time: 5e-05 seconds

Obstacle Density: High
No path found
Nodes Explored: 1
Execution Time: 2e-05 seconds